DOCUMENTAÇÃO TÉCNICA

TESTIFY + MONEYFLOW

Biblioteca de testes em Go e sua aplicação prática

Conceitos • Estrutura do projeto • Exemplos • Execução

Sumário

1. Apresentação e objetivo
2. Contexto: Go e testes automatizados
3. Gerenciamento do projeto Go: go.mod e go.sum
4. Visão geral do MoneyFlow
5. Organização dos arquivos
6. Testify: recursos utilizados
7. Mock e Fake no MoneyFlow
8. Exemplos de cenários testados
9. Como executar o projeto
10. Benefícios e pontos de atenção
11. Relação com JUnit
12. Conclusão
13. Roteiro sugerido para os slides
14. Referências

1. Apresentação e objetivo

Esta documentação reúne os conceitos da biblioteca Testify e sua aplicação prática no projeto MoneyFlow. O objetivo é apresentar não apenas o que cada recurso faz, mas também como ele foi utilizado para verificar as regras de negócio de uma aplicação de controle de despesas.

O MoneyFlow é um projeto demonstrativo desenvolvido em Go. Ele permite cadastrar despesas, consultar o saldo disponível, listar o histórico e emitir uma notificação quando o orçamento é totalmente consumido. O projeto utiliza armazenamento em memória, sem banco de dados ou comunicação HTTP.

A documentação está organizada para servir como material de consulta e como base para a elaboração de slides e para a apresentação oral.

2. Contexto: Go e testes automatizados

Go (também chamado de Golang) é a linguagem utilizada para implementar o MoneyFlow. A linguagem possui um mecanismo nativo de testes por meio do pacote testing. Os testes normalmente ficam em arquivos terminados em _test.go e são escritos em funções cujo nome começa com Test.

```
func TestExemplo(t *testing.T) {
    // Arrange: preparar os dados
    // Act: executar a funcionalidade
    // Assert: verificar o resultado
}
```

O comando go test compila e executa os testes encontrados. O Testify não substitui o pacote testing: ele é uma biblioteca complementar que oferece funções de verificação, simulação de dependências e organização de testes.

Comandos utilizados

```
go test ./...          # executa os testes dos pacotes
go test -v ./...       # apresenta detalhes da execução
go test -cover ./...   # apresenta a cobertura do código
```

A cobertura indica quais partes do código foram exercitadas pelos testes. Um percentual alto, sozinho, não garante que os cenários importantes estejam bem verificados.

3. Gerenciamento do projeto Go: go.mod e go.sum

O arquivo go.mod identifica o módulo Go e declara a versão de Go de referência e as dependências diretas do projeto. No MoneyFlow, o módulo é chamado moneyflow e a dependência principal de testes é github.com/stretchr/testify, na versão 1.10.0.

```
module moneyflow

go 1.22

require github.com/stretchr/testify v1.10.0
```

O arquivo go.sum registra hashes de verificação associados às versões dos módulos baixados. Esses registros ajudam o Go a verificar a integridade do conteúdo das dependências. Ele não substitui o go.mod: os dois arquivos têm funções diferentes e devem acompanhar o código-fonte.

- go.mod: informa qual é o módulo e quais dependências e versões são necessárias.
- go.sum: registra verificações de integridade das dependências.

O comando go mod tidy pode ser utilizado para ajustar as dependências declaradas e os registros de soma conforme os imports do projeto.

4. Visão geral do MoneyFlow

O MoneyFlow controla despesas a partir de um limite de orçamento definido na criação do Service. Os valores monetários são armazenados em centavos (int64), evitando imprecisões comuns em operações com ponto flutuante.

Fluxo principal de criação de uma despesa

1. Os dados recebidos são normalizados e validados.
2. O serviço consulta as despesas já registradas para calcular o total gasto.
3. O serviço calcula o saldo disponível e verifica se a nova despesa cabe no orçamento.
4. Se a regra for atendida, a despesa recebe um identificador e é enviada ao repositório.
5. Se a despesa consumir exatamente todo o orçamento, o serviço aciona o notificador de forma assíncrona.

Se os dados forem inválidos, o orçamento for ultrapassado ou ocorrer uma falha na consulta ou gravação, o serviço retorna um erro e não conclui a operação.

Principais componentes

- Expense: estrutura que representa uma despesa, com identificador, descrição, valor em centavos e categoria.
- Repository: interface que define as operações Save e List usadas para persistência.
- MemoryRepository: implementação em memória da interface Repository.
- Notifier: interface que abstrai o envio de notificações.
- Service: coordena validação, cálculo do orçamento, gravação e notificação.
- cmd/demo: aplicação de terminal que permite demonstrar o fluxo de uso.

5. Organização dos arquivos

Arquivo	Responsabilidade
expense.go	Define Expense, erros de domínio, validação e criação de despesas.
repository.go	Define Repository e implementa MemoryRepository, que armazena despesas em memória.
service.go	Implementa as regras de negócio: validação, saldo, limite, persistência e notificação.
mocks_test.go	Implementa MockRepository e MockNotifier com testify/mock.
expense_test.go	Testa validações e estrutura Expense usando assert e require; também verifica o repositório em memória.
service_mock_test.go	Testa o Service com dependências simuladas, verificando retornos e interações.
service_suite_test.go	Agrupa testes do Service em uma suíte e combina fake e mock.
cmd/demo/main.go	Contém a demonstração interativa executada no terminal.
go.mod / go.sum	Gerenciam o módulo e a integridade das dependências Go.

A separação entre código de produção e arquivos _test.go mantém os testes próximos ao código, mas fora do fluxo normal de execução da aplicação.

6. Testify: recursos utilizados

6.1. assert — verificar sem interromper imediatamente

O pacote assert oferece funções para comparar resultados e verificar condições. Quando uma asserção falha, o teste é marcado como falho, mas as próximas linhas do teste podem continuar sendo executadas. Isso é útil quando várias verificações independentes podem fornecer informações sobre o mesmo cenário.

```
assert.Equal(t, "Café", got.Description)
assert.Equal(t, "alimentacao", got.Category)
assert.Equal(t, int64(850), got.AmountCents)
```

No MoneyFlow, assert é utilizado para verificar campos normalizados, valores, erros esperados, conteúdo de listas, identificadores e resultados do serviço.

6.2. require — interromper quando a condição é essencial

O pacote require oferece verificações semelhantes às de assert, mas interrompe o teste atual quando uma condição falha. É indicado para pré-condições. Por exemplo, se uma operação deveria retornar sem erro antes de permitir a análise do objeto, require.NoError evita que o teste continue usando um resultado inválido.

```
got, err := moneyflow.NewExpense(" Café ", 850, "Alimentacao")
require.NoError(t, err)
assert.Equal(t, "Café", got.Description)
```

No exemplo, require.NoError garante que a criação foi bem-sucedida. Depois disso, assert verifica os campos retornados.

6.3. mock — controlar dependências

O pacote mock permite programar o comportamento de uma dependência. O teste define qual método pode ser chamado, com quais argumentos, quantas vezes e quais valores devem ser retornados. Depois, verifica se as interações esperadas aconteceram.

```
f.repo.On("List", ctx).Return(history, nil).Once()
f.repo.On("Save", ctx, want).Return(nil).Once()

got, err := f.svc.CreateExpense(ctx, "Almoço", 1990, "alimentacao")

require.NoError(t, err)
assert.Equal(t, want, got)
f.repo.AssertExpectations(t)
```

O teste não chama diretamente os métodos do mock como objetivo final. Ele injeta os mocks no Service real e executa CreateExpense. Assim, verifica a regra de negócio e também se o Service interagiu corretamente com as dependências.

6.4. suite — organizar testes relacionados

O pacote suite permite reunir testes em uma struct que incorpora suite.Suite. Essa estrutura pode manter dependências compartilhadas e definir métodos de preparação e finalização.

- SetupSuite: executa uma vez antes dos testes da suíte.
- SetupTest: executa antes de cada teste.
- SetupSubTest: prepara cada subteste executado com s.Run.
- TearDownTest: executa ao final de cada teste.
- TearDownSuite: executa uma vez ao final da suíte.

No MoneyFlow, ServiceSuite agrupa cenários do Service. O método reset cria um novo repositório em memória, um notificador mockado e um novo Service para manter os cenários isolados. O pacote suite não oferece suporte a testes paralelos com t.Parallel().

7. Mock e Fake no MoneyFlow

Mock e Fake são tipos de test doubles (substitutos de teste), mas não significam a mesma coisa. Mock é um recurso oferecido pelo Testify no pacote mock. Fake é um conceito geral de testes e, neste projeto, foi implementado manualmente.

MockRepository e MockNotifier

Os mocks incorporam mock.Mock e usam m.Called(...) para registrar a chamada e recuperar os valores configurados pelo teste. Com On(...).Return(...), o teste programa a resposta. Com Once, Times ou Maybe, controla a quantidade de chamadas esperadas. Com AssertExpectations, verifica se as expectativas foram atendidas.

MemoryRepository como Fake

MemoryRepository implementa a interface Repository e mantém uma coleção de despesas em memória. Ele executa operações de Save e List de maneira funcional, mas sem banco de dados. Por isso, é classificado como Fake: possui comportamento simplificado, porém funcional.

Tipo	No projeto	Finalidade

Mock	MockRepository e MockNotifier	Controlar respostas e verificar chamadas e argumentos.
Fake	MemoryRepository	Executar operações reais simplificadas em memória.

Importante: Fake não é um pacote do Testify. O Testify fornece o pacote mock; o MemoryRepository foi escrito no próprio projeto.

8. Exemplos de cenários testados

- Normalização: remove espaços desnecessários da descrição e padroniza a categoria.
- Validação: rejeita valor zero, valor negativo e descrição vazia.
- Orçamento excedido: rejeita uma despesa maior que o saldo disponível e verifica que Save não foi chamado.
- Falha no repositório: simula erros na consulta ou gravação e verifica se o erro original continua identificável.
- Identificadores: verifica a geração sequencial exp-1, exp-2 e assim por diante.
- Notificação: verifica que o aviso é enviado quando o orçamento é totalmente consumido e não é enviado enquanto há saldo.
- Histórico e saldo: verifica se despesas aceitas aparecem no histórico e alteram o saldo corretamente.

A suíte inclui também um cenário de uso de 33,33% do orçamento, utilizando InDelta para comparar valores decimais com uma tolerância.

9. Como executar o projeto

Executar a demonstração interativa

```
go run ./cmd/demo
```

O programa de terminal demonstra o controle de despesas com orçamento de R$ 100,00. É possível adicionar despesas, consultar o saldo e visualizar o histórico.

Executar os testes

```
go test ./... -v
go test -cover ./...
```

O primeiro comando executa os testes com saída detalhada. O segundo apresenta a cobertura dos pacotes.

Executar um teste ou método específico

```
go test -run TestMustNewExpense -v
go test -run TestServiceSuite -testify.m TestUsedPercent -v
```

O segundo exemplo seleciona o método TestUsedPercent dentro da suíte TestServiceSuite.

10. Benefícios e pontos de atenção

Benefícios observados

- Legibilidade: assert e require tornam as verificações mais diretas.
- Isolamento: mocks permitem testar o Service sem depender de uma implementação externa de persistência ou notificação.
- Controle: as expectativas dos mocks ajudam a verificar se métodos foram chamados com os argumentos e a frequência corretos.
- Organização: suite reúne cenários relacionados e centraliza a preparação dos testes.
- Reprodutibilidade: o repositório em memória permite executar cenários sem configurar um banco de dados.

Pontos de atenção

- Testify é uma dependência adicional e precisa ser declarada e mantida no módulo.
- Os cenários e resultados esperados precisam ser definidos por quem desenvolve; a biblioteca não encontra todos os defeitos automaticamente.
- Cobertura de código não é sinônimo de qualidade dos testes.
- Mocks verificam interações programadas; seu uso excessivo pode deixar testes muito dependentes da implementação.
- A suíte do Testify não deve ser combinada com t.Parallel().

A demonstração utiliza armazenamento em memória e não representa uma solução completa de produção com banco de dados, autenticação ou API.

11. Relação com JUnit

JUnit é uma ferramenta de testes amplamente utilizada no ecossistema Java. Testify é uma biblioteca complementar para Go. As duas ferramentas apoiam testes automatizados, mas pertencem a linguagens e ecossistemas diferentes.

No Go, o pacote testing já oferece a estrutura fundamental de testes. Testify acrescenta recursos para assertions, mocks e suites. Portanto, a comparação é pela finalidade geral, e não por equivalência exata entre APIs ou arquitetura.

12. Conclusão

O MoneyFlow demonstra como Testify pode ser aplicado em um projeto Go para validar regras de negócio e interações entre componentes. assert e require verificam resultados em momentos diferentes do teste; mock permite controlar dependências e conferir chamadas; suite organiza cenários relacionados.

O projeto também mostra a diferença entre um Mock, configurado para responder de acordo com as expectativas do teste, e um Fake, como o MemoryRepository, que executa operações simplificadas em memória.

A principal contribuição do Testify é tornar os testes mais expressivos e organizados. A qualidade final, porém, depende da escolha de bons cenários, da clareza das verificações e da manutenção dos testes.

13. Roteiro sugerido para os slides

6. Tema e objetivo: o que é Testify e por que foi utilizado no MoneyFlow.
7. Contexto de Go: pacote testing, arquivos _test.go e comando go test.
8. O projeto MoneyFlow: objetivo, fluxo de criação de despesas e componentes.
9. Arquivos e dependências: organização do projeto, go.mod e go.sum.
10. assert e require: diferença de continuidade e exemplos do projeto.
11. mock: como configurar expectativas, executar o Service e verificar chamadas.
12. Fake x Mock: MemoryRepository comparado aos mocks do Testify.

13. suite: struct, ciclo de vida e isolamento dos cenários.
14. Cenários testados e demonstração: validação, orçamento, falhas e notificação.
15. Conclusão: benefícios, limitações e aprendizados.

14. Referências

- Testify — repositório oficial: https://github.com/stretchr/testify
- Testify — documentação geral: https://pkg.go.dev/github.com/stretchr/testify
- Testify — assert: https://pkg.go.dev/github.com/stretchr/testify/assert
- Testify — require: https://pkg.go.dev/github.com/stretchr/testify/require
- Testify — mock: https://pkg.go.dev/github.com/stretchr/testify/mock
- Testify — suite: https://pkg.go.dev/github.com/stretchr/testify/suite
- Go — pacote testing: https://pkg.go.dev/testing
- Go — documentação sobre testes: https://go.dev/doc/code#Testing